



EST160 - Aula 02

Lucas Sousa

18 de Agosto de 2026

Conteúdos

Projetos Reprodutíveis

Estrutura de Script

Revisão

Introdução

Como cientistas de dados, é importante que possamos estruturar projetos de forma organizada e reprodutível. Isso vai além de um código organizado;

Introdução

Como cientistas de dados, é importante que possamos estruturar projetos de forma organizada e reproduzível. Isso vai além de um código organizado;

O *script* não é autossuficiente, muitas vezes dependendo de arquivos que devem estar em algum lugar;

Introdução

Como cientistas de dados, é importante que possamos estruturar projetos de forma organizada e reproduzível. Isso vai além de um código organizado;

O *script* não é autossuficiente, muitas vezes dependendo de arquivos que devem estar em algum lugar;

Também devemos ter o cuidado de não deixar os códigos muito grandes, tentando separá-los em seções;

Introdução

Como cientistas de dados, é importante que possamos estruturar projetos de forma organizada e reprodutível. Isso vai além de um código organizado;

O *script* não é autossuficiente, muitas vezes dependendo de arquivos que devem estar em algum lugar;

Também devemos ter o cuidado de não deixar os códigos muito grandes, tentando separá-los em seções;

Além disso, precisamos armazenar os resultados de nossas análises de forma organizada.

Introdução

Devemos nos perguntar:

“Se eu entregar essa pasta, projeto ou código para alguém, a pessoa vai conseguir reproduzir ou compreender o que fiz?”

Introdução

Algumas etapas da organização do projeto são:

- ▶ Criação de uma estrutura clara de diretórios (como as pastas estarão organizadas);
- ▶ Divisão de scripts por função;
- ▶ Organização dos códigos
- ▶ Garantir autossuficiência do código
- ▶ Documentação

Introdução

Reprodutibilidade

Autores providenciam todos os dados necessários, além dos códigos adequados para se realizar uma análise de dados, recriando todos os resultados.

Introdução

Reprodutibilidade

Autores providenciam todos os dados necessários, além dos códigos adequados para se realizar uma análise de dados, recriando todos os resultados.

Replicabilidade

É possível utilizar o mesmo código para realizar análises adequadas a partir de um conjunto de dados diferente, mas com estrutura semelhante ao do projeto dos autores.

Exemplo

Se eu mandasse esse código para vocês, ele funcionaria?

```
setwd("C:/Users/Joao/Documents/Projeto_final/
      dados")

dados <- read_csv("dados.csv")

dados_limpos <- dados |>
  filter(idade > 18)

write_csv(
  dados_limpos,
  "C:/Users/Joao/Desktop/resultado_final.csv"
)
```

Exemplo

- Onde estão os dados?
- Você também se chama Joao e tem o mesmo caminho no computador?
- A coluna “idade” sequer existe?
- Todas as funções são nativas do R?

```
setwd("C:/Users/Joao/Documents/Projeto_final/
      dados")
```

```
dados <- read_csv("dados.csv")
```

```
dados_limpos <- dados |>
  filter(idade > 18)
```

```
write_csv(
  dados_limpos,
  "C:/Users/Joao/Desktop/resultado_final.csv"
)
```

"mAs FuNcloNoU nO mEu
cOMpuTaDoR!"



Introdução

Com um projeto organizado, seus colegas podem

- ▶ verificar a análise;
- ▶ corrigir um erro;
- ▶ utilizar o código;
- ▶ adaptar o método;
- ▶ produzir uma nova análise.

Introdução

Com um projeto organizado, seus colegas podem

- ▶ verificar a análise;
- ▶ corrigir um erro;
- ▶ utilizar o código;
- ▶ adaptar o método;
- ▶ produzir uma nova análise.

Se mais gente trabalha no mesmo projeto, isso torna viável a **co-laboração**, que geralmente é feita por meio de repositórios como *GitHub*.

Exemplo

Execute

```
x <- 10
```

Exemplo

Execute

```
x <- 10
```

Agora, atribua e chame

```
resultado <- x + 5  
resultado
```

Exemplo

Execute

```
x <- 10
```

Agora, atribua e chame

```
resultado <- x + 5  
resultado
```

Recemos como saída

```
> 15
```

Agora, em uma sessão recém-aberta, atribua

```
resultado <- x + 5
```

Exemplo

Execute

```
x <- 10
```

Agora, atribua e chame

```
resultado <- x + 5  
resultado
```

Recemos como saída

```
> 15
```

Agora, em uma sessão recém-aberta, atribua

```
resultado <- x + 5
```

Recebemos como saída

Erro: variável x não está definida.

Por quê?

Ambiente

O ambiente (*environment*) registra as variáveis e dados que temos em nossa seção.

É possível salvar o ambiente ao fechar o R, mas não podemos depender disso para organizar os projetos

Ambiente

O ambiente (*environment*) registra as variáveis e dados que temos em nossa seção.

É possível salvar o ambiente ao fechar o R, mas não podemos depender disso para organizar os projetos

É possível que seu projeto funcione por causa de **dependências invisíveis**, isto é, dados salvos no ambiente

Use `ls()` para verificar o ambiente.

Working Directory (Diretório de Trabalho)

A função `setwd()` determina o **caminho absoluto** até o diretório de trabalho atual

Chamamos de caminho absoluto porque precisamos dizer explicitamente o caminho até a pasta que queremos

Working Directory (Diretório de Trabalho)

A função `setwd()` determina o **caminho absoluto** até o diretório de trabalho atual

Chamamos de caminho absoluto porque precisamos dizer explicitamente o caminho até a pasta que queremos

Gostaríamos de trabalhar com **caminhos relativos**, a partir da raiz do projeto

Exemplo:

```
read_csv("data/dados.csv")
```

Organizem-se!

Mesmo com caminhos relativos, pode ser difícil ou confuso chamar o diretório se não houver organização.

```
meu-projeto/  
├── data/  
│   └── dados.csv  
├── scripts/  
│   └── limpeza.R  
└── outputs/  
    └── resultado.csv
```

Nesse caso, `read_csv("data/dados.csv")` nos diz

“Entre na pasta data que está dentro do diretório atual, e leia dados.csv.”

Organizem-se!

Podemos ter outra estrutura.

```
raiz/
├── data/
│   └── dados.csv
├── meu-projeto/
│   └── limpeza.R
└── outputs/
    └── resultado.csv
```

Nesse caso, `read_csv("../data/dados.csv")` nos diz

“Suba uma pasta, entre na pasta data e leia dados.csv.”

Organizem-se!

Queremos evitar

```
"../../../../data/dados.csv"
```

Uma alternativa é utilizar o pacote `library(here)`, que parte da raiz do projeto.

```
library(here)
dados <- read_csv(
  here("data", "dados.csv")
)
```

Mesmo assim, é importante ter uma organização adequada.

Compare Organizações

Projeto A

projeto/

- dados.csv
- dados2.csv
- dados_final.csv
- tabela.csv
- grafico.png
- grafico2.png
- analise.R
- analise2.R
- teste.R
- final.R

Projeto B

projeto/

- data/
 - raw/
 - processed/
- R/
- scripts/
- outputs/
 - figures/
 - tables/
- report/
- README.md
- projeto.Rproj

Organizações

Queremos poder responder rapidamente:

- ▶ Onde estão os dados brutos?
- ▶ Onde estão os dados processados?
- ▶ Onde estão os scripts?
- ▶ Onde estão os resultados?
- ▶ Onde está a documentação?

Estrutura Detalhada

Temos uma pasta mãe chamada projeto. Dentro dessa pasta, organizaremos nosso projeto.

Em data/raw/ colocamos os dados brutos (crus), e idealmente **não modificamos manualmente.**

Em data/processed/ colocamos os dados processados **através de scripts**

Estrutura Detalhada

Temos uma pasta mãe chamada `projeto`. Dentro dessa pasta, organizaremos nosso projeto.

Em `scripts/` colocamos os códigos executáveis, preferencialmente separados por objetivo.

Exemplo:

`01-importacao.R`

`02-limpeza.R`

`03-analise.R`

`04-visualizacao.R`

Estrutura Detalhada

Temos uma pasta mãe chamada projeto. Dentro dessa pasta, organizaremos nosso projeto.

Em outputs/ colocaremos as saídas de nossos códigos (gráficos e tabelas)

Preferencialmente organize essas pastas, principalmente se a análise tiver diversas etapas ou objetivos.

Estrutura Detalhada

Temos uma pasta mãe chamada `projeto`. Dentro dessa pasta, organizaremos nosso projeto.

Crie um arquivo `README.md` que idealmente deve responder

O que é o projeto?

Qual é o objetivo?

Onde estão os dados?

Como executar e quais pacotes são necessários?

Quem participou?

Como citar?

Existem limitações ou restrições?

Isso é bastante recomendável para utilizar *GitHub*.

Evite

Dá vontade, mas evite:

analise-final.R

analise-final2.R

analise-final-agora-vai.R

aaaaaa.R

qpoksqs kjd.R

Evite

Dá vontade, mas evite:

analise-final.R

analise-final2.R

analise-final-agora-vai.R

aaaaaa.R

qpoksqs kjd.R

Utilize nomes legíveis por máquinas e por pessoas.

Projetos

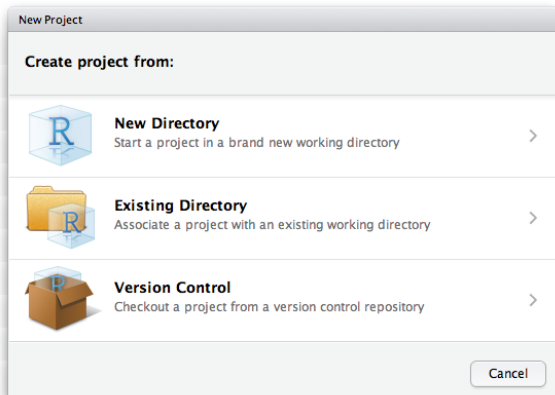
No R podemos criar um novo projeto, ou abrir um projeto já existente utilizando o arquivo em formato `.Rproj`. Mais detalhes nesse link.

Também é possível acessar um projeto por meio do *GitHub* (Controle de Versão), onde o alteramos **localmente** antes de enviar para o site.

Projetos

No R podemos criar um novo projeto, ou abrir um projeto já existente utilizando o arquivo em formato `.Rproj`. Mais detalhes nesse [link](#).

Também é possível acessar um projeto por meio do *GitHub* (Controle de Versão), onde o alteramos **localmente** antes de enviar para o site.



Estrutura do Script

A estrutura do *script* também é importante para uma organização do projeto. Isso inclui mais do que comentar o código.

Como mencionado, recomenda-se a separação de códigos para diferentes etapas do projeto, de modo que não tenhamos um único código gigante.

Estrutura do Script

A estrutura do *script* também é importante para uma organização do projeto. Isso inclui mais do que comentar o código.

Como mencionado, recomenda-se a separação de códigos para diferentes etapas do projeto, de modo que não tenhamos um único código gigante.

"A função estava na linha 2319,
como não encontrou?"



Exemplo

```
##### INFO #####

# PROJETO
## Trabalho/Projeto: O TÍTULO DO SEU TRABALHO/
  PROJETO ## Autores: SEU NOME

# R Script
## Descrição: Esse script performa uma análise
  descritiva sobre em dados sobre (______). Os
  dados podem ser encontrados em (______).
  As variáveis de interesse são;
# Var_1: descrição
# Var_2: descrição
# Var_3: descrição
## Inputs: data_raw/dados.csv
## Outputs: Data_processed/output.csv

# Observações:
```

Exemplo

```
# Setup ----

# install.packages("tidyverse")
# install.packages("here")
# install.packages("modelsummary")

library(tidyverse)
library(here)
library(modelsummary)

# Estabelece raiz para caminho relativo
i_am("Projeto/3_main_analysis.R")

# Leitura dos dados brutos ----

nome_data <- read_csv(
  here("data_raw", "dados.csv")
)
```

Exemplo

```
# Análise Principal ----

modelo <- lm(leave ~ turnout + income + noqual,
             data = dados)

# Saída do resuom do modelo ----

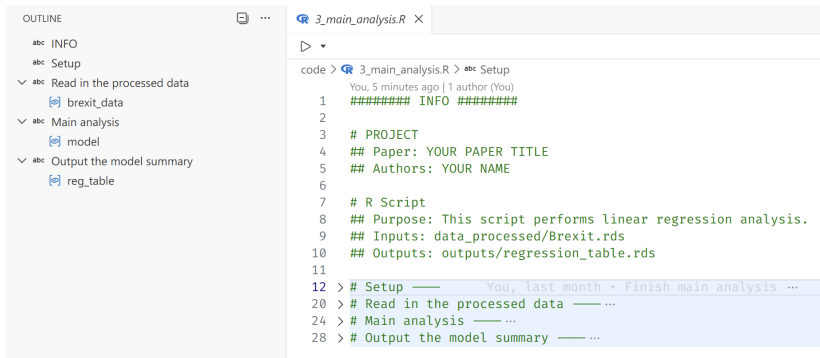
tab_reg <- modelsummary(modelo, stars = TRUE,
                        output = "latex")
write_csv(tab_reg, here("outputs/output.csv"))
```

Organizem-se!

Notem que

```
# SECTION ----
```

cria uma estrutura se seções no RStudio, e isso é útil para projetos maiores.



The screenshot shows the RStudio interface. On the left, the 'OUTLINE' pane displays a hierarchical structure of the project files:

- abc INFO
- abc Setup
- abc Read in the processed data
 - brexit_data
- abc Main analysis
 - model
- abc Output the model summary
 - reg_table

On the right, the '3_main_analysis.R' script is open in the editor. The code is as follows:

```
code > 3_main_analysis.R > abc Setup
You, 5 minutes ago | 1 author (You)
1 ##### INFO #####
2
3 # PROJECT
4 ## Paper: YOUR PAPER TITLE
5 ## Authors: YOUR NAME
6
7 # R Script
8 ## Purpose: This script performs linear regression analysis.
9 ## Inputs: data_processed/Brexit.rds
10 ## Outputs: outputs/regression_table.rds
11
12 > # Setup — You, last month • Finish main analysis ...
20 > # Read in the processed data — ...
24 > # Main analysis — ...
28 > # Output the model summary — ...
```

Evite

É importante comentar o **porquê** de algumas coisas, mas evite comentários vagos, ou registros que podem ser apagados.

O **terminal** poder utilizado para testes, chamar tabelas, instalar pacotes, etc.

Você pode ter sripts exploratórios, mas evite compartilhar versões prolixas.

Evite

```
x <- read_csv("teste.csv")

# testar coisa
x <- x |> filter(a > 2)

# não funcionou

x <- read_csv("teste2.csv")

# agora acho que é isso
x <- x |> filter(a > 3)

# teste
x <- x |> filter(a > 4)
```

Evite

```
x <- read_csv("teste.csv")  
  
# testar coisa  
x <- x |> filter(a > 2)  
  
# não funcionou  
  
x <- read_csv("teste2.csv")  
  
# agora acho que é isso  
x <- x |> filter(a > 3)  
  
# teste  
x <- x |> filter(a > 4)
```

Atentem-se à lógica

Na computação, existe um passo a passo lógico para toda ação.

Atentem-se à lógica

Na computação, existe um passo a passo lógico para toda ação.

Muitas vezes os erros ocorrem por nomes incorretos, dados inexistentes, erros com símbolos e falta de compreensão das funções utilizadas.

Atentem-se à lógica

Na computação, existe um passo a passo lógico para toda ação.

Muitas vezes os erros ocorrem por nomes incorretos, dados inexistentes, erros com símbolos e falta de compreensão das funções utilizadas.

É normal recebemos erros, mas evitem gambiarras. Entendam o código.

Revisão

Para calcular uma média, podemos pensar nos seguintes passos:

1. obter os valores;
2. somar os valores;
3. contar quantos valores existem;
4. dividir a soma pela quantidade;
5. apresentar o resultado.

Revisão

Para calcular uma média, podemos pensar nos seguintes passos:

1. obter os valores;
2. somar os valores;
3. contar quantos valores existem;
4. dividir a soma pela quantidade;
5. apresentar o resultado.

A função `mean()` faz todo esse trabalho.

É comum existir uma função mais otimizada para o que queremos fazer. Melhorar o código é um exercício.

Revisão

```
x <- c(10, 12, 15, 13, 20) # x é um vetor  
mean(x) # mean() é uma função nativa do R  
> [1] 10 # Essa é a saída
```

Estrutura condicional

```
idade <- 20

if (idade >= 18) {
  "adulto"
} else {
  "menor de idade"
}
```

Aqui, temos uma ordem lógica:

- ▶ se a condição for atendida, imprima "adulto";
- ▶ senão, imprima "menor de idade";

Para entender isso, a linguagem usa **lógica Booleana**.

Lógica Booleana

Analisaremos operadores **lógicos** que irão nos retornar valores **verdadeiros** ou **falso**.

Lógica Booleana

Analisaremos operadores **lógicos** que irão nos retornar valores **verdadeiros** ou **falso**.

```
idade >= 18
```

produz um resultado lógico:

```
> TRUE
```

ou

```
> FALSE
```

Operadores Lógicos

Operador	Significado
>	maior que
<	menor que
>=	maior ou igual a
<=	menor ou igual a
==	igual a
!=	diferente de
&	E
	OU
!	NÃO

Operadores Lógicos

Operador	Significado
>	maior que
<	menor que
>=	maior ou igual a
<=	menor ou igual a
==	igual a
!=	diferente de
&	E
	OU
!	NÃO

Note que:

```
x <- 10      # atribuição
x == 10      # comparação
```

Repetição

A função `for()` aplica uma ação para CADA objeto do conjunto

Repetição

A função `for()` aplica uma ação para CADA objeto do conjunto

```
for (x in 1:5) {  
  print(x)  
}
```

```
> 1  
> 2  
> 3  
> 4  
> 5
```

Repetição

A função `while()` aplica uma ação ENQUANTO uma condição não for atendida (ou até que seja atendida)

Repetição

A função `while()` aplica uma ação ENQUANTO uma condição não for atendida (ou até que seja atendida)

É normal criarmos um loop infinito nesse caso. **CUIDADO!** O programa só para quando a condição for atendida, então é importante garantir que um dia será.

Repetição

A função `while()` aplica uma ação ENQUANTO uma condição não for atendida (ou até que seja atendida)

É normal criarmos um loop infinito nesse caso. **CUIDADO!** O programa só para quando a condição for atendida, então é importante garantir que um dia será.

While

```
i <- 0  # atribua um valor a uma variável

# crie a repetição
while (i <= 9) {
  print(paste("ainda não chegou! seu resultado é
              ", i))
}
```

```
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 0"
```

While

```
i <- 0  # atribua um valor a uma variável

# crie a repetição
while (i <= 9) {
  print(paste("ainda não chegou! seu resultado é
              ", i))
  i = i + 1
}
```

```
[1] "ainda não chegou! seu resultado é 0"
[1] "ainda não chegou! seu resultado é 1"
[1] "ainda não chegou! seu resultado é 2"
[1] "ainda não chegou! seu resultado é 3"
[1] "ainda não chegou! seu resultado é 4"
[1] "ainda não chegou! seu resultado é 5"
[1] "ainda não chegou! seu resultado é 6"
[1] "ainda não chegou! seu resultado é 7"
[1] "ainda não chegou! seu resultado é 8"
[1] "ainda não chegou! seu resultado é 9"
```

Variáveis

Númericas (num)

```
idade <- 35  
altura <- 1.75
```

Texto/Strings (str)

```
nome <- "Maria"  
username <- "141414"
```

Lógica (log)

```
aprovado <- TRUE  
reprovado <- FALSE
```

Variáveis

O valor NA representa uma informação ausente ou desconhecida.

```
idade <- c(20, 25, NA, 32)
```

NA não significa zero. Significa que não temos uma informação válida para aquela observação.

Acessando vetores

Podemos criar um vetor utilizando a função `c()`:

```
idades <- c(20, 25, 31, 19, 42)
```

Acessando vetores

Podemos criar um vetor utilizando a função `c()`:

```
idades <- c(20, 25, 31, 19, 42)
```

Podemos acessar os elementos individualmente

```
idades[1]
```

```
> [1] 20
```

```
idades[3]
```

```
> [1] 31
```

Acessando vetores

Podemos criar um vetor utilizando a função `c()`:

```
idades <- c(20, 25, 31, 19, 42)
```

Podemos acessar os elementos individualmente

```
idades[1]
```

```
> [1] 20
```

```
idades[3]
```

```
> [1] 31
```

Também podemos acessar um intervalo:

```
idades[2:4]
```

```
> [1] 25 31 19
```

Acessando vetores

Podemos criar um vetor utilizando a função `c()`:

```
idades <- c(20, 25, 31, 19, 42)
```

Podemos acessar os elementos individualmente

```
idades[1]  
> [1] 20
```

```
idades[3]  
> [1] 31
```

Também podemos acessar um intervalo:

```
idades[2:4]  
> [1] 25 31 19
```

Ou selecionar elementos de acordo com uma condição:

```
idades[idades > 25]  
> [1] 31 42
```

Operando Vetores

Podemos somar valores números a vetores. Todas as entradas somam o valor.

```
x <- c(10, 20, 30, 40)
x + 5
> [1] 15 25 35 45
```

Operando Vetores

Podemos somar valores números a vetores. Todas as entradas somam o valor.

```
x <- c(10, 20, 30, 40)
x + 5
> [1] 15 25 35 45
```

Da mesma maneira:

```
x * 2
> [1] 20 40 60 80
```

Operando Vetores

Podemos somar valores números a vetores. Todas as entradas somam o valor.

```
x <- c(10, 20, 30, 40)
x + 5
> [1] 15 25 35 45
```

Da mesma maneira:

```
x * 2
> [1] 20 40 60 80
```

Se temos também `y <- c(2, 4, 3, 8)`, podemos fazer

```
x / y
> [1] 5 5 10 5
```

Obs: os vetores precisam ter o mesmo tamanho!

Algumas funções

```
mean(x)  
median(x)  
sd(x)  
min(x)  
max(x)  
sum(x)  
length(x)
```

Algumas funções

```
mean(x)  
median(x)  
sd(x)  
min(x)  
max(x)  
sum(x)  
length(x)
```

Considerando

```
mean(x, na.rm = TRUE)
```

- `mean()` é a função;
- `x` é o objeto;
- `na.rm = TRUE` é um argumento.

Estruturas de dados

Um vetor armazena uma sequência de valores.

```
x <- c(10, 20, 30)
```

Estruturas de dados

Um vetor armazena uma sequência de valores.

```
x <- c(10, 20, 30)
```

Uma lista pode conter diferentes objetos.

```
pessoa <- list(  
  nome = "Ana",  
  idade = 30,  
  notas = c(8, 9, 7)  
)
```

```
> pessoa  
$nome  
[1] "Ana"
```

```
$idade  
[1] 30
```

```
$notas  
[1] 8 9 7
```

Estruturas de dados

Também temos uma estrutura de `data.frame`

```
dados <- data.frame(  
  idade = c(20, 25, 30),  
  sexo = c("F", "M", "F"),  
  nota = c(8.5, 7.2, 9.1)  
)
```

```
> dados  
  idade sexo nota  
1    20   F  8.5  
2    25   M  7.2  
3    30   F  9.1
```

As colunas podem ser vistas como vetores.

```
dados[1, 2]  
dados$idade  
dados[, 2]
```

Operador Pipe

O operador pipe `|>` permite representar uma sequência de operações de maneira mais próxima da leitura natural.

```
dados |>  
  subset(idade >= 25)
```

Operador Pipe

O operador pipe `|>` permite representar uma sequência de operações de maneira mais próxima da leitura natural.

```
dados |>  
  subset(idade >= 25)
```

Podemos interpretar como:

“Pegue dados e use a saída na próxima operação.”

Operador Pipe

O operador pipe `|>` permite representar uma sequência de operações de maneira mais próxima da leitura natural.

```
dados |>  
  subset(idade >= 25)
```

Podemos interpretar como:

“Pegue dados e use a saída na próxima operação.”

E isso é equivalente a

```
> subset(dados, idade >= 25)
```

ambos nos dão

	idade	sexo	nota
2	25	M	7.2
3	30	F	9.1